IE4483 Sentiment Analysis Project Plan

Oct 27, 2026 · @bryan

Overview

The goal is to build and compare three sentiment classifiers on product reviews, then submit one group report and `submission.csv` to NTULearn by Nov 13, 2026 (11:59pm).

- **Task:** classify each review as negative (0) or positive (1). Train on `train.json`, predict on the test set (1,851 rows in the `submission.csv` template).
- **Group:** three students, one report that names each member's contribution.
- **Starting point:** the current `submission.csv` has every prediction set to 0, so it is an unfilled template.
- **Three models, each adding one capability:** TF-IDF + Logistic Regression (word counts, no order) → GloVe + BiLSTM (full sequence) → fine-tuned BERT (pretrained context).

Q	What the report must cover	Marks
a	Feature format and preprocessing steps	10%
b	Models: architecture figure, input/output dimensions, loss, training strategy, code and run instructions	20%
c	How and why each parameter was chosen (learning rate and others)	10%
d	`submission.csv` with test-set predictions	20%
e	Correct and incorrect samples, 1-2 cases on strengths and weaknesses	10%
f	Feature format vs resource use and accuracy	10%
g	Hotel reviews with no rating scores	10%
h	Hotel reviews with noisy rating scores, three improvements	10%

Data and shared preprocessing

All three models share one split and one base cleaning, then each gets its own feature extraction, so differences in results come from the model and not from the data handling.

Data. `train.json` holds `reviews` (raw text) and `sentiments` (0 negative, 1 positive); the product reviews come from the Blitzer et al. (2007) multi-domain dataset. Check the row count, class balance, nulls, duplicates and the review-length distribution before anything else, and keep the plots for the report.

Shared steps (identical for every model):

1. Strip HTML tags and normalize whitespace.
2. Drop empty and duplicate reviews.
3. Make one stratified train/validation split (for example 80/20) with a fixed seed, and save it to disk.
4. Fit anything data-dependent (TF-IDF vocabulary, GloVe vocabulary) on the training split only, to avoid leakage.

Model-specific feature extraction (different on purpose):

Model	Input form	Key choices
TF-IDF + LR	Sparse TF-IDF vector	Lowercase, unigrams + bigrams, `max_features`, keep negations ("not", "no", "never")
GloVe + BiLSTM	Sequence of GloVe vectors	Lowercase, tokenize, vocabulary, pad or truncate to a length chosen from the length histogram
BERT	WordPiece token ids + attention mask	Minimal cleaning only: no stopword removal, no lemmatizing, keep punctuation and casing as the tokenizer expects

The report should state this explicitly for (a): the base cleaning is shared, and feature extraction differs because each model expects a different input.

Model 1: TF-IDF + Logistic Regression

This is the cheap, interpretable baseline: it turns each review into word-weight counts and learns one weight per word, so it sets the score every other model must beat. Typical accuracy on binary review sentiment is roughly 86-90% (an expectation, not a guarantee).

How it works

1. **TF-IDF** gives each word (or word pair) a weight: how often it appears in this review (term frequency), scaled down if it appears in most reviews (inverse document frequency). Common words like "the" end up near zero.
2. Each review becomes one long sparse vector with a weight per vocabulary entry.
3. **Logistic regression** multiplies that vector by learned weights, adds a bias, and passes the sum through a sigmoid to give P(positive). Words with large positive weights ("excellent") push toward positive; large negative weights ("waste") push toward negative.
4. Training minimizes cross-entropy loss with L2 regularization, controlled by `C`.

$$P(y=1 \mid x) = \sigma(w^{\top} x + b), \qquad \mathcal{L} = -\sum_i \big[y_i \log p_i + (1-y_i)\log(1-p_i) \big] + \tfrac{1}{2C}\lVert w \rVert^2$$

Why include it: it is fast (seconds on CPU), its weights show which words drive predictions (useful for error analysis in (e)), and it gives a first valid `submission.csv` early.

Limits: it ignores word order, so "not good" and "good, not bad" are hard for it; bigrams help only partly.

Shapes: input `(n_reviews, max_features)` sparse; output `(n_reviews,)` probability, thresholded at 0.5.

What to tune: n-gram range (1,1) vs (1,2); `max_features`; `min_df` / `max_df`; sublinear TF; regularization strength `C`; compare Logistic Regression with a linear SVM as a side check.

Model 2: GloVe + BiLSTM (with attention)

This model reads the review as an ordered sequence in both directions, so it can handle negation and context that bag-of-words misses. Typical accuracy is roughly 87-91%.

How it works

1. Words become GloVe vectors, giving `(seq_len, embed_dim)`.
2. An **LSTM** reads the vectors one at a time and keeps a hidden state with gates that decide what to remember, forget and output. This lets it carry "not" forward to the word it modifies.
3. A **bidirectional** LSTM runs one pass left to right and one right to left, then concatenates the two states at each position, so each word sees context from both sides.

4. **Attention** (optional) scores each position and takes a weighted sum, so sentiment-heavy words count more than filler. Without attention, use the final hidden states or max-pooling.
5. The review vector goes through dropout and a linear layer to give two class scores; training minimizes cross-entropy.

Why include it: it is the cheapest model that uses word order, so it tests whether sequence information helps beyond TF-IDF.

Limits: slower to train than TF-IDF + LR, can overfit on small data, and struggles with very long reviews; its embeddings are static ("sick" has one vector in every sentence).

Shapes: input `(batch, seq_len)` → embeddings `(batch, seq_len, embed_dim)` → BiLSTM output `(batch, seq_len, 2 × hidden)` → pooled `(batch, 2 × hidden)` → output `(batch, 2)`.

What to tune: learning rate first; then hidden size, number of layers, dropout, gradient clipping, frozen vs fine-tuned embeddings, and early stopping.

Model 3: Fine-tuned BERT

BERT starts from a language model already trained on large text corpora and is fine-tuned on your reviews, so it brings contextual word meaning that the other three models must learn from scratch. Typical accuracy is roughly 92-95%, and it should be the main model for (b) and the final `submission.csv`.

How it works

1. The **WordPiece tokenizer** splits text into subword tokens and adds `[CLS]` at the start and `[SEP]` at the end; reviews are truncated or padded to `max_length` with an attention mask.
2. Token, position and segment embeddings are summed and passed through **12 transformer encoder layers** (BERT-base). **Self-attention** lets every token look at every other token, so "bank" gets a different vector in "river bank" and "bank loan", and "not" can directly modify "good".
3. The final vector of `[CLS]` summarizes the review. It goes through dropout and a linear layer (768 → 2) to give two class scores.
4. **Fine-tuning** updates all layers with a small learning rate, using cross-entropy, AdamW, linear warmup then decay, and usually 2-4 epochs.

Why include it: pretraining gives it language knowledge from far more text than the training set, so it usually wins on small and medium datasets.

Limits: about 110M parameters, needs a GPU for reasonable speed, is sensitive to learning rate and seed, and truncates long reviews. DistilBERT (about 40% smaller, close in accuracy) is the fallback if time or GPU is short.

Shapes: input ids and attention mask `(batch, max_length)` → hidden states `(batch, max_length, 768)` → `[CLS]` vector `(batch, 768)` → output `(batch, 2)`.

What to tune: learning rate first (2e-5, 3e-5, 5e-5); then epochs, warmup ratio, weight decay, batch size and `max_length` (pick from the length histogram); use early stopping and 3 seeds for the final run.

Experiment protocol and optimization plan

Every model is tuned on the same validation split with the same protocol, and the final numbers come from 3 seeds, so the comparison is fair and the report can show evidence for each choice (this is what questions (b), (c) and (f) reward).

Fair-comparison rules

1. One saved stratified train/validation split for all models; the test set is never used for tuning.
2. Tune one parameter at a time, most impactful first (learning rate), then the rest; no full grid.
3. Single-seed runs during tuning; rerun the best configuration with 3 seeds and report mean ± std.
4. Log every run in one table: model, settings, validation accuracy, macro-F1, training time, peak memory.
5. The tuning budget can differ by model (light for TF-IDF, focused for BERT), but state it.

Metrics: accuracy, macro-F1 and a confusion matrix; optionally McNemar's test to check whether BERT's gain over the baseline is significant. Also record training time, inference time, parameter count and memory for (f).

Optimization deep dive (for (c))

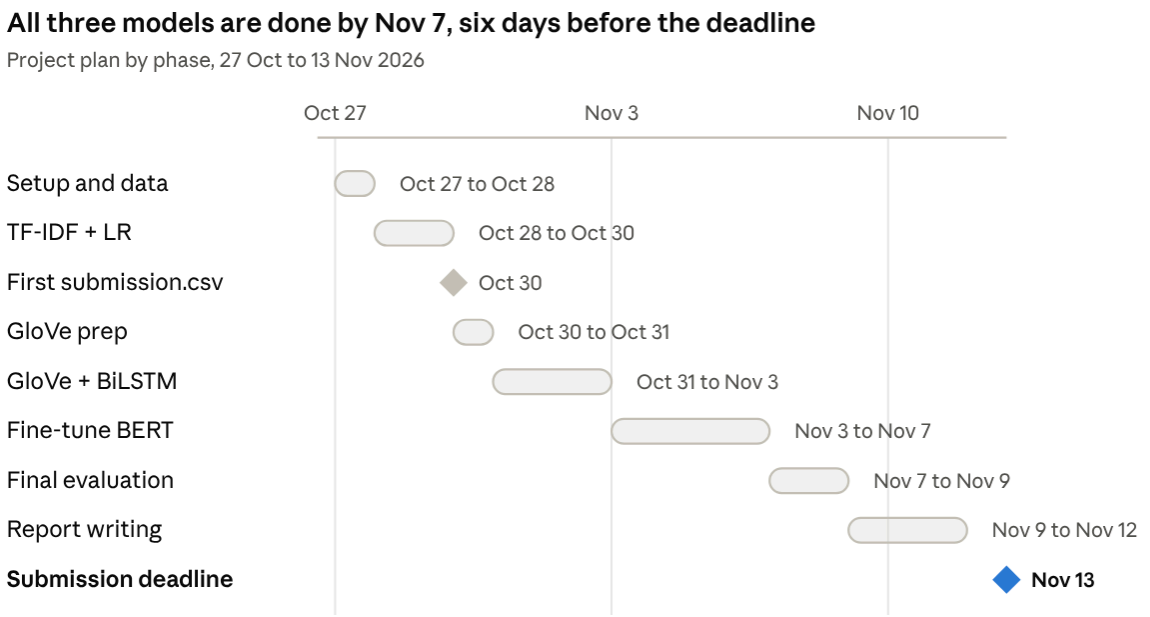
Topic	What to run	What to show
Learning rate	Sweep per model (BERT 2e-5, 3e-5, 5e-5; BiLSTM 1e-4 to 1e-2)	Validation accuracy vs learning rate; explain divergence when too high and slow or stuck when too low
Scheduler and warmup	Constant vs linear decay vs warmup + decay	Training curves; why warmup stabilizes transformers

Topic	What to run	What to show
Optimizer	Adam vs AdamW vs SGD	Convergence speed; why AdamW (decoupled weight decay) suits BERT
Epochs and early stopping	Train past the optimum	Train and validation loss curves with the chosen epoch marked
Regularization	Dropout, weight decay, frozen vs fine-tuned embeddings	Gap between train and validation accuracy
Batch size and sequence length	Two or three values each	Accuracy vs memory and time trade-off
Seeds	3 seeds for finals	Mean ± std, so real gains are separated from noise

Ablations worth one small table each: TF-IDF unigram vs bigram; GloVe frozen vs fine-tuned; BERT with vs without warmup. For each setting, write what was tried, what was measured and what was concluded.

Resource measurement for (f): record wall-clock training time, inference time per 1,000 reviews, parameter count and peak GPU or RAM for all three models on the same hardware, and discuss accuracy gained per unit of cost.

TODO plan and timeline



project timeline · 7 phases, 2 milestones

All three models are finished by Nov 7, which leaves six days before the 13 Nov deadline; work through the phases in order and tick items off as they are done.

Phase 1: Setup and data (Oct 27-28)

- [ ] Create the project folder and `requirements.txt`; set one global random seed
- [ ] Load `train.json` and `test.json`; check columns, row counts, nulls and duplicates
- [ ] Exploratory analysis: class balance, length histogram, sample reviews; save the plots
- [ ] Apply the shared base cleaning
- [ ] Make and save the stratified train/validation split
- [ ] Write the shared evaluation function and the run log

Phase 2: TF-IDF + Logistic Regression (Oct 28-30)

- [ ] Build TF-IDF on the training split only, keeping negation words
- [ ] Train a default model and log validation scores
- [ ] Tune n-gram range, `max_features` and `C`
- [ ] Export the first `submission.csv` as a safe early version

Phase 3: GloVe preparation (Oct 30-31)

- [ ] Download GloVe; build the vocabulary and embedding matrix from the training split
- [ ] Choose the sequence length from the length histogram; write the padding code

Phase 4: GloVe + BiLSTM (Oct 31 - Nov 3)

- [ ] Build the BiLSTM (with attention if time allows) and train a default version
- [ ] Tune learning rate first, then hidden size, dropout and frozen vs fine-tuned embeddings; add early stopping
- [ ] Save loss curves and the best checkpoint

Phase 5: Fine-tuned BERT (Nov 3-7)

- [ ] Tokenize with the BERT tokenizer (minimal cleaning) and choose `max_length`
- [ ] Fine-tune a default setup, then sweep the learning rate (2e-5, 3e-5, 5e-5), then epochs and warmup
- [ ] Save loss curves and the best checkpoint

Phase 6: Final evaluation (Nov 7-9)

- [ ] Rerun each model's best configuration with 3 seeds; record mean ± std
- [ ] Measure training time, inference time, memory and parameter count for all three
- [ ] Error analysis on the validation set; pick 1-2 cases for (e)
- [ ] Choose the best model and generate the final `submission.csv`; check 1,851 rows, values 0/1, same columns and row order as the template

Phase 7: Report (Nov 9-12)

- [ ] Make the figures: architecture diagrams, loss curves, learning-rate plots, comparison table
- [ ] Write sections (a) to (h) following the report structure
- [ ] Add the contribution table, references and the appendix with code and run instructions
- [ ] Proofread, test the run instructions, and submit on Nov 12 if possible

Fallbacks if time slips

- Use DistilBERT instead of BERT-base if the GPU is slow.
- Skip BiLSTM attention if it takes too long.
- Shorten the tuning sweeps if needed, but keep the 3-seed runs for the final models.

Report structure

The report follows the question order (a) to (h), so the marker finds each answer under its own heading, and the comparison tables carry most of the detail while the text explains what the results mean.

Section	Content	Marks
Title page	Group members and each member's contribution	required
Introduction	Task, dataset, approach in one page	-
(a) Data and preprocessing	Exploratory plots, shared base cleaning, model-specific features and why	10%
(b) Models	Per model: why chosen, architecture figure, dimensions, loss, training strategy; reproducibility (seeds, runs) and how to run the code	20%
(c) Hyperparameter tuning	Protocol, one subsection per model with sweep plots and tables, final-settings table with reasons	10%
(d) Results and submission	Multi-seed comparison table, chosen model, how `submission.csv` was generated and checked	20%

Section	Content	Marks
(e) Error analysis	Confusion matrix; 1-2 cases showing a strength and a weakness	10%
(f) Feature format vs resources	Accuracy vs training time, inference time, memory and parameters for all three	10%
(g) Hotel reviews, no ratings	Expected performance and modifications	10%
(h) Hotel reviews, noisy ratings	Expected performance and three improvements	10%
Conclusion and references	Main findings, limits; Blitzer 2007, BERT, GloVe, Word2Vec and any libraries; appendix with code and run instructions	-

Notes for specific sections

- **(e) Test labels are probably hidden,** so do the correct and incorrect analysis on the validation set, and say so in the report. For the test set, inspect low-confidence predictions instead. Good cases: negation ("not bad"), sarcasm, mixed sentiment, and a long review cut at `max_length`.
- **(g) No ratings means domain shift and no labels.** Hotel vocabulary ("staff", "noisy", "clean") differs from product vocabulary, and TF-IDF should degrade more than BERT. Ideas: zero-shot use of the current model checked on a small hand-labeled sample, continued pretraining on unlabeled hotel text, pseudo-labeling, a sentiment lexicon for weak labels, and active learning on the most uncertain reviews.
- **(h) Noisy ratings make large models memorize wrong labels.** Pick three approaches: a robust loss (label smoothing, symmetric or generalized cross-entropy), label cleaning with confident learning (for example cleanlab), and a training strategy such as early stopping, co-teaching or ensembling; check each on a small clean validation set.
- **Honour Code and citations:** cite every source, credit adapted code, and write the analysis in the group's own words.